@Transactional: Propagation, Isolation and Traps

7 min read Updated 28 Jul 2026 Topic 59 of 65

Bookmark Mark done

Why it matters

`@Transactional` is one annotation with a remarkable number of ways to silently do nothing — self-invocation, private methods, checked exceptions, the wrong propagation. Every one of those has caused a production data-integrity bug somewhere, and interviewers ask precisely because the failures are silent: no exception, no log, just data that should have rolled back and did not.

The concept, simply

`@Transactional` wraps a method in a database transaction via the AOP proxy from Chapter 11: begin before, commit after, roll back on failure.

The default rollback rule is the trap: Spring rolls back on **unchecked** exceptions and `Error`, and **commits** on checked exceptions. That surprises almost everyone — a method that throws `IOException` after half its work commits the half.

JAVA Copy

```java
@Transactional
public void process() throws IOException {
    repository.save(order);
    throw new IOException("failed");      // ✗ COMMITS — checked exception
}

@Transactional(rollbackFor = Exception.class)    // ✓ roll back on anything
public void process() throws IOException { … }
```

How it works

Propagation decides what happens when a transactional method is called from within an existing transaction:

Propagation	Behaviour
`REQUIRED` (default)	Join the existing one, or start one
`REQUIRES_NEW`	Suspend the outer, run in a genuinely independent transaction
`SUPPORTS`	Join if present, otherwise run without
`MANDATORY`	Must already be in one, else throw
`NEVER` / `NOT_SUPPORTED`	Throw / suspend and run without
`NESTED`	Savepoint within the outer — rolls back to the savepoint

`REQUIRES_NEW` **is the one with a real use case:** writing an audit record that must survive even when the business transaction rolls back. It is also the one that causes connection-pool exhaustion, because the outer transaction holds its connection while the inner one takes a second — nest that under load and the pool is gone.

Isolation controls what a transaction can see of others' uncommitted or changing data — `READ_COMMITTED` is the usual default and prevents dirty reads; `REPEATABLE_READ` additionally prevents non-repeatable reads; `SERIALIZABLE` prevents phantoms at a significant concurrency cost. In practice the default is almost always right, and correctness problems are better solved with optimistic locking.

Code examples

The four silent failures, together:

JAVA

Copy

```java
@Service
public class OrderService {
```

```
    public void processAll(List<Order> os) { os.forEach(this::processOne); }
    @Transactional public void processOne(Order o) { … }   // NO transacti

    // ✗ 2. PRIVATE — proxies cannot intercept it. Silently ignored.
    @Transactional private void internal() { … }

    // ✗ 3. CHECKED EXCEPTION — commits instead of rolling back.
    @Transactional public void save() throws IOException { … }

    // ✗ 4. CATCHING THE EXCEPTION — no exception propagates, so Spring ha
    //      no idea anything failed and commits happily.
    @Transactional
    public void process() {
        try { risky(); }
        catch (Exception e) { log.error("failed", e); }    // ✗ still comm
    }
    // ✓ Rethrow, or mark the transaction rollback-only:
    //   TransactionAspectSupport.currentTransactionStatus().setRollbackOn
}
```

Optimistic locking, which is the answer to most concurrency questions here:

JAVA

Copy

```java
@Entity
public class Order {
    @Version
    private Long version;          // ✓ Hibernate manages it
}

// Every UPDATE includes "WHERE id = ? AND version = ?" and increments it.
// If another transaction updated the row first, zero rows match and
// Hibernate throws OptimisticLockException — no locks held, no deadlocks,
// and the loser can retry.

@Retryable(retryFor = OptimisticLockingFailureException.class, maxAttempts
@Transactional
public void updateStatus(Long id, OrderStatus status) { … }

// Pessimistic locking takes a real database lock and blocks other writers
```

```
// Correct, and far more likely to deadlock — reserve it for genuine
// ...ot row, and always with a lock ordering (Chapter 8).
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT o FROM Order o WHERE o.id = :id")
Optional<Order> findByIdForUpdate(@Param("id") Long id);
```

Under the hood

Why checked exceptions commit is a design decision, not an oversight: Spring inherited EJB's convention that checked exceptions are *business* outcomes the caller is expected to handle, while unchecked ones are failures. Whether or not you agree, knowing it is deliberate is a better answer than calling it a bug — and `rollbackFor = Exception.class` is the fix.

The transaction spans the whole method, including slow work. An HTTP call inside a transactional method holds a database connection open for its entire duration. Under load that exhausts the pool while the database itself is idle. Keep transactions short and do external calls outside them — the same discipline as "never hold a lock across I/O" from Chapter 8.

Transaction boundaries belong in the service layer, not the repository and not the controller. Repository-level transactions make a multi-step business operation non-atomic; controller-level ones hold connections while serialising responses. The service method is the unit of business work, which is exactly the unit that should be atomic.

Common mistakes

- **Self-invocation.** The most common cause of a silently missing transaction.
- `@Transactional` **on a private method.** Cannot be proxied; ignored.
- **Expecting a rollback on a checked exception.** It commits — use `rollbackFor`.
- **Catching the exception inside the method.** Spring sees success and commits.
- **External calls inside a transaction.** Holds a connection for the whole round trip.
- `REQUIRES_NEW` **in a loop.** Each takes another connection — pool exhaustion.
- **Raising the isolation level to fix a race.** Optimistic locking with `@Version` is usually better.

- **Transactions on repositories or controllers.** Boundaries belong in the service layer.

Interview lens

Q: "How does `@Transactional` work?"

A: Through the AOP proxy. Spring wraps the bean, and when an external caller invokes an annotated method the proxy begins a transaction, delegates, then commits or rolls back. Because it is proxy-based, it shares every proxy limitation: a self-invocation bypasses it, a `private` or `final` method cannot be intercepted, and in both cases it fails silently.

Q: "When does Spring roll back by default?"

A: On unchecked exceptions and `Error` only. A checked exception *commits*, which is a deliberate inheritance from EJB semantics — checked exceptions were treated as business outcomes rather than failures. If you want a checked exception to roll back, specify `rollbackFor = Exception.class`. And catching the exception inside the method means Spring sees no failure at all and commits.

Q: "Explain `REQUIRED` versus `REQUIRES_NEW`."

A: `REQUIRED`, the default, joins an existing transaction or starts one — so an inner failure rolls back the whole thing. `REQUIRES_NEW` suspends the outer transaction and runs in a genuinely independent one, so it can commit even when the outer rolls back. That is exactly right for audit logging, and dangerous in a loop, because the outer transaction keeps its connection while each inner one takes another.

Q: "Two users update the same order concurrently. How do you handle it?"

A: Optimistic locking. Add a `@Version` field; Hibernate then includes the version in the update's `WHERE` clause and increments it, so the second writer matches zero rows and gets an `OptimisticLockException` to retry. It holds no locks and cannot deadlock. Pessimistic locking with `PESSIMISTIC_WRITE` is the alternative when

contention on a specific row is genuinely high, but it blocks writers and risks deadlock.

Q: "Why did my transaction not roll back?"

A: Four usual suspects, in order of likelihood. Self-invocation, so the proxy was never involved. A checked exception, which commits by default. The exception being caught inside the method, so Spring saw success. Or the method being `private` or `final` and therefore un-proxyable. All four are silent — nothing is logged, which is what makes them so hard to spot.

Q: "Where should transaction boundaries go?"

A: The service layer, because that is where a unit of business work lives and that is what needs to be atomic. On repositories, a multi-step operation stops being atomic. On controllers, the connection is held while the response is serialised. I also keep transactions short and move any external HTTP call outside them, since a transactional method holds a database connection for its entire duration.

Cheat-card

- Proxy-based ⇒ **self-invocation,** `private` **and** `final` **all silently skip it**.
- **Rolls back on unchecked only.** Checked exceptions COMMIT → `rollbackFor = Exception.class`.
- Catching the exception inside ⇒ Spring commits. Rethrow or `setRollbackOnly()`.
- Propagation: `REQUIRED` (default) · `REQUIRES_NEW` (independent, audit) · `NESTED` (savepoint) · `MANDATORY` · `SUPPORTS` · `NEVER`.
- `REQUIRES_NEW` in a loop ⇒ **connection-pool exhaustion**.
- Concurrency: `@Version` **optimistic locking** first; pessimistic only for hot rows.
- **No external calls inside a transaction** — it holds a connection throughout.
- Boundaries in the **service layer**. Read paths get `readOnly = true`.

1. By default, when does Spring roll back a transaction?

- ◯ Only when you call setRollbackOnly()
- ◯ On checked exceptions only
- ◯ On any exception
- ◯ On unchecked exceptions and Error ONLY — a checked exception COMMITS

Check answer

2. What is REQUIRES_NEW for, and what is its risk?

- ◯ It suspends the outer transaction and runs independently — right for audit records that must survive a rollback — but each nested call takes a SECOND connection, so using it in a loop exhausts the pool
- ◯ It makes the transaction read-only; the risk is stale data
- ◯ It disables rollback entirely
- ◯ It retries the transaction on failure; the risk is duplicate writes

Check answer

3. Two users update the same row concurrently. What is the standard JPA solution?

- ◯ Raise the isolation level to SERIALIZABLE
- ◯ Optimistic locking with a @Version field — the UPDATE includes the version in its WHERE clause, so the second writer matches zero rows and gets an exception it can retry

Synchronize the service method

- ◯ Use REQUIRES_NEW so each update is independent

Check answer

‹ Previous
Lazy Loading and the N+1 Problem

Next ›
REST API Design in Spring

LEARN MORE

ARTICLES & DOCS

- Transaction management — Spring Framework reference
  docs.spring.io
- Declarative transaction management
  docs.spring.io

VIDEOS



Transaction Management in hibernate in spring boot Interview questions | with Example | Code Decode
Code Decode



Hibernate N+1 problem and solution | Hibernate Interview Questions and Answers | Code Decode
Code Decode

Hand-picked references for this lesson.